1. Introducción

La Programación Orientada a Objetos (POO) es un paradigma que organiza el software en objetos: unidades que reúnen datos y el comportamiento que opera sobre ellos. En vez de escribir una secuencia de instrucciones sobre datos sueltos, el programa se modela como un conjunto de objetos que colaboran entre sí, de forma parecida a cómo funcionan las entidades del mundo real.

Este documento explica los fundamentos de la POO y por qué es la base adecuada para un sistema que debe crecer. Los ejemplos provienen del Gestor Inteligente de Clientes (GIC), la plataforma desarrollada para la startup SolutionTech, que hoy administra sus clientes de forma manual y sufre duplicación de datos, errores y problemas de seguridad.

Python soporta la POO de forma nativa: todo en Python es un objeto, incluidos los números, las listas y las funciones. Por eso el paradigma se integra de manera natural con el resto del lenguaje.

2. Conceptos base

Concepto	Definición	En el GIC
Clase	Plantilla que define qué datos y qué comportamiento tendrá un tipo de objeto	`Cliente`, `Direccion`, `GestorClientes`
Objeto (instancia)	Elemento concreto creado a partir de una clase, con sus propios valores	El cliente Ana Rojas, RUT 11.111.111-1
Atributo	Dato que describe el estado de un objeto	`rut`, `email`, `telefono`, `direccion`
Método	Función definida dentro de la clase que describe lo que el objeto puede hacer	`calcular_descuento()`, `mensaje_bienvenida()`
Constructor	Método `__init__` que se ejecuta al crear el objeto e inicializa sus atributos	Valida RUT, email y teléfono al instanciar
`self`	Referencia al objeto que está ejecutando el método	`self.nombre` es el nombre de ese cliente en particular

Una misma clase genera muchos objetos independientes. Cambiar el email de un cliente no afecta a los demás, porque cada objeto guarda su propio estado.

```
class Direccion:
    def __init__(self, calle, numero, comuna, region):   # constructor
        self.calle = calle                                  # atributos
        self.numero = numero
        self.comuna = comuna
        self.region = region


    def __str__(self):                                      # método
        return f"{self.calle} {self.numero}, {self.comuna}"
```

```python
# Instanciación: dos objetos de la misma clase
casa = Direccion("Av. Libertador", "456", "Rancagua", "O'Higgins")
oficina = Direccion("Av. Providencia", "1650", "Providencia", "Metropolitana")

print(casa)      # Av. Libertador 456, Rancagua
print(oficina)   # Av. Providencia 1650, Providencia
```

3. Los cuatro pilares de la POO

Abstracción

Abstraer es representar solo lo esencial de una entidad y ocultar los detalles que no importan para el problema. Un cliente real tiene cientos de características, pero a SolutionTech solo le interesan su identificación, contacto, dirección y tipo.

En el GIC, `Cliente` es una clase abstracta: define qué debe saber hacer todo cliente, sin decir cómo. Python impide crear un `Cliente` genérico; solo se pueden crear sus tipos concretos.

```python
from abc import ABC, abstractmethod

class Cliente(ABC):
    @abstractmethod
    def calcular_descuento(self, monto: float) -> float:
        """Cada tipo de cliente define su propia regla."""
```

Encapsulación

Encapsular es proteger el estado interno de un objeto y permitir modificarlo solo por vías controladas. Así ningún código externo puede dejar al objeto con datos inválidos.

En el GIC los atributos son privados (doble guion bajo) y se acceden mediante `@property`. El setter valida antes de guardar:

```python
@property
def email(self):
    return self.__email

@email.setter
def email(self, valor):
    self.__email = validar_email(valor)   # rechaza "correo-sin-arroba"
```

El RUT, además, es de solo lectura: una vez creado el cliente, su identificador no se puede cambiar.

Herencia

La herencia permite crear una clase nueva a partir de otra existente, reutilizando sus atributos y métodos y agregando o modificando lo necesario. Se expresa como una relación “es un”: un cliente premium es un cliente.

`ClienteRegular`, `ClientePremium` y `ClienteCorporativo` heredan de `Cliente`. Con `super()` reutilizan toda la validación del padre y solo agregan lo propio:

```
class ClientePremium(Cliente):
    def __init__(self, *args, nivel="plata", puntos=0, **kwargs):
        super().__init__(*args, **kwargs)   # RUT, email, teléfono, dirección
        self.nivel = nivel
        self.puntos = puntos
```

Polimorfismo

Polimorfismo significa “muchas formas”: un mismo mensaje produce respuestas distintas según el objeto que lo recibe. El código que llama no necesita saber con qué tipo concreto está trabajando.

```
for cliente in [ana, pedro, carla]:          # regular, premium, corporativo
    print(cliente.calcular_descuento(100_000))
# 0.0      -> regular: sin descuento
# 15000.0  -> premium oro: 15%
# 7000.0   -> corporativo de 120 empleados: 5% + 2%
```

Pilar	Pregunta que responde	Beneficio principal
Abstracción	¿Qué es esencial modelar?	Reduce la complejidad
Encapsulación	¿Quién puede cambiar los datos y cómo?	Protege la integridad de los datos
Herencia	¿Qué comportamiento se puede reutilizar?	Evita duplicar código
Polimorfismo	¿Cómo tratar distintos tipos de forma uniforme?	Permite extender sin modificar

4. Enfoque procedural vs orientado a objetos

El mismo cálculo de descuento muestra la diferencia. En un enfoque procedural, los datos viven en diccionarios y una función central decide qué hacer con cada tipo:

```
# Procedural: los datos y las reglas están separados
cliente = {"tipo": "premium", "nivel": "oro", "email": "pedro@correo.cl"}

def calcular_descuento(cliente, monto):
    if cliente["tipo"] == "regular":
        return 0
    elif cliente["tipo"] == "premium":
        tasas = {"plata": 0.10, "oro": 0.15, "platino": 0.20}
        return monto * tasas[cliente["nivel"]]
    elif cliente["tipo"] == "corporativo":
        return monto * min(0.05 + cliente["empleados"] // 50 * 0.01, 0.25)

cliente["email"] = "correo-malo"   # nada lo impide
```

En POO, cada tipo de cliente sabe calcular su propio descuento y protege sus datos:

```
# Orientado a objetos: cada clase es dueña de sus datos y sus reglas
pedro = ClientePremium(..., nivel="oro")
```

```
pedro.calcular_descuento(100_000)   # 15000.0
pedro.email = "correo-malo"         # lanza EmailInvalidoError
```

Aspecto	Procedural	Orientado a objetos
Organización	Funciones y datos separados	Datos y comportamiento juntos en clases
Protección de datos	Cualquier parte del código puede modificar un diccionario	Los setters validan cada cambio
Agregar un tipo nuevo	Editar cada `if/elif` repartido en el código	Crear una subclase nueva
Riesgo al modificar	Alto: un cambio puede romper funciones lejanas	Bajo: el cambio queda dentro de una clase
Pruebas	Difícil aislar una regla	Cada clase se prueba por separado
Adecuado para	Scripts cortos y tareas puntuales	Sistemas que crecen y se mantienen en el tiempo

El enfoque procedural no es incorrecto: para un script de 50 líneas es más simple. Pero a medida que el sistema suma tipos, reglas y personas trabajando en él, las cadenas de `if/elif` se multiplican y cada cambio se vuelve riesgoso. Ese es justamente el problema de escalabilidad que la POO resuelve.

5. Importancia de la POO en sistemas escalables

Un sistema es escalable cuando puede crecer en funciones, datos, usuarios o equipo de desarrollo sin tener que reescribirse. La POO no hace que un programa sea más rápido, pero sí hace que crecer sea seguro y barato. Contribuye en cuatro dimensiones:

Dimensión	Problema al crecer	Cómo ayuda la POO	Ejemplo en el GIC
Funcional	Nuevos requerimientos obligan a tocar código que ya funciona	Herencia y polimorfismo permiten extender sin modificar	Un nuevo tipo de cliente es una subclase más
Técnica	Cambiar de tecnología implica reescribir	La abstracción separa el qué del cómo	`GestorClientes` depende de `RepositorioClientes`, no de SQLite
Mantenimiento	Un cambio de regla está repartido en muchos lugares	La encapsulación concentra cada regla en un solo punto	La regla del teléfono vive solo en `validar_telefono()`
Equipo	Varias personas editan el mismo código y se pisan	Clases con responsabilidades claras se reparten y prueban por separado	Una persona trabaja la interfaz Flask y otra la persistencia

Principios SOLID

Los principios SOLID son cinco guías de diseño orientado a objetos que mantienen el código flexible a medida que crece. El GIC los aplica así:

Principio	Significado	Aplicación en el GIC
S: Responsabilidad única	Cada clase tiene un solo motivo para cambiar	`Notificador` solo envía correos; `ValidadorIdentidad` solo valida; `RepositorioSQLite` solo guarda
O: Abierto/cerrado	Abierto a extensión, cerrado a modificación	Se agregan tipos de cliente sin editar `Cliente` ni `GestorClientes`
L: Sustitución de Liskov	Una subclase puede reemplazar a su clase padre sin romper nada	Donde se espera un `Cliente` funciona cualquiera de los tres tipos
I: Segregación de interfaces	Interfaces pequeñas y específicas	`RepositorioClientes` define solo las operaciones de datos de clientes
D: Inversión de dependencias	Depender de abstracciones, no de implementaciones	El repositorio, validador y notificador se inyectan en el constructor del gestor

Inyección de dependencias y pruebas

Como `GestorClientes` recibe sus colaboradores desde afuera, en producción usa SQLite, la API real y SMTP, mientras que en las pruebas recibe una base temporal y objetos simulados:

```
# Producción
gestor = GestorClientes(RepositorioSQLite("data/gic.db"),
                        ValidadorIdentidad(), Notificador())

# Pruebas: mismo gestor, sin internet ni correos reales
gestor = GestorClientes(RepositorioSQLite("/tmp/prueba.db"),
                        ValidadorIdentidad(habilitado=False), MagicMock())
```

Esto permitió escribir 50 pruebas unitarias que corren en menos de un segundo. Un sistema que se puede probar rápido se puede cambiar con confianza, y esa confianza es la que permite que crezca.

6. Caso práctico: agregar un nuevo tipo de cliente

Supongamos que SolutionTech empieza a atender a municipalidades y necesita un `ClienteGobierno` con número de licitación y un descuento fijo de 12%. Gracias a la POO, el cambio se resuelve con una clase nueva y una línea en la fábrica:

```
class ClienteGobierno(Cliente):
    TIPO = "gobierno"

    def __init__(self, *args, licitacion: str = "", **kwargs):
        super().__init__(*args, **kwargs)
```

```
        self.licitacion = validar_texto(licitacion, "licitación", 5, 30)

    def calcular_descuento(self, monto):
        return round(monto * 0.12, 2)

    def beneficios(self):
        return ["Descuento institucional 12%", "Facturación con orden de compra"]

    def to_dict(self):
        return {**super().to_dict(), "licitacion": self.licitacion}


TIPOS = {c.TIPO: c for c in (ClienteRegular, ClientePremium,
                             ClienteCorporativo, ClienteGobierno)}
```

El impacto en el resto del sistema:

Componente	¿Requiere cambios?	Motivo
`Cliente` (clase base)	No	La nueva clase solo la extiende
Validaciones de RUT, email y teléfono	No	Se heredan con `super()`
`GestorClientes`	No	Trabaja con cualquier `Cliente` (polimorfismo)
`RepositorioSQLite`	No	Los campos extra se guardan en la columna `extra` como JSON
Exportación JSON	No	Usa `to_dict()` de cada objeto
`Notificador`	No	Llama a `mensaje_bienvenida()`, que usa los nuevos `beneficios()`
Formulario web	Mínimo	Agregar el campo de licitación en la plantilla
Pruebas existentes	No	Siguen pasando; se agregan pruebas para la nueva clase

Con un enfoque procedural, el mismo cambio exigiría encontrar y editar cada `if tipo == ...` del cálculo de descuentos, los beneficios, el correo de bienvenida, el guardado y la exportación, con el riesgo de olvidar alguno. Esta diferencia en el costo del cambio es lo que hace a la POO la base de los sistemas escalables.

7. Conclusión

La POO organiza el software en objetos que protegen sus datos, reutilizan comportamiento y responden de forma uniforme aunque sean de tipos distintos. Sus cuatro pilares atacan directamente los problemas que SolutionTech quiere resolver: la encapsulación evita datos inválidos, la abstracción y la herencia eliminan código duplicado, y el polimorfismo permite agregar funciones sin romper las existentes.

En sistemas escalables, el valor de la POO está en reducir el costo del cambio. Como muestra el caso de `ClienteGobierno`, un requerimiento nuevo se resuelve agregando código en vez de modificar el que ya funciona. Combinada con principios como SOLID y con pruebas unitarias, la POO permite que el GIC crezca en tipos de cliente, tecnologías e integraciones manteniendo un código ordenado y confiable.